

Locally Self-Supervised Deep Learning for Vision

CS39440 Major Project Report

Author: Alessio Pomo (alp104@aber.ac.uk)

Supervisor: Dr. Bernie Tiddeman (bpt@aber.ac.uk)

15th April 2026

Version: 1.0 (Release)

This report was submitted as partial fulfilment of a
BSc degree in Computer Science (G400)



Department of Computer Science, Aberystwyth University
Aberystwyth, Ceredigion, SY23 3DB, Wales, U.K.

Declaration of Originality

I confirm that:

- This submission is my own work, except where clearly indicated.
- I understand that there are severe penalties for Unacceptable Academic Practice, which can lead to loss of marks or even the withholding of a degree.
- I have read the regulations on Unacceptable Academic Practice from the University's Academic Registry (AR) and the relevant sections of the current Student Handbook of the Department of Computer Science.
- I have read the document Generative AI and the Major and Minor Projects, which is available in the Blackboard module.
- In submitting this work I understand and agree to abide by the University's regulations governing these issues.

Name: Alessio Pomo

Date: 15/04/2026

Consent to share this work

By including my name below, I hereby agree to this project's report and technical work being made available to other students and academic staff of the Aberystwyth Computer Science Department.

Name: Alessio Pomo

Date: 15/04/2026

Acknowledgements

Thanks to Dr. Tiddeman for all the assistance throughout the duration of the project.

AI Declaration

GitHub's Copilot autofill function has been used when writing the code. This was activated as a plug-in in Visual Studio Code. Whilst writing the code, Copilot made suggestions for completing blocks such as if statements and for statements and a lot more simpler replacements. These were accepted in a lot of situations to make life easier.

Abstract

Neural systems in biology do not learn through global error backpropagation. However, this is still the most widely used training method for artificial deep learning. This project explores alternatives to this method based on locally self-supervised learning architecture for computer vision, taking inspiration from the hierarchical and local nature of biological neural processing.

Over the course of this project a progressive development along with deeper understanding of more complex artificial networks is demonstrated, starting off from only simple Convolutional Neural Networks all the way to researching and delving into the most novel approaches in tackling the challenge that this project is centered around, just another clash between the evergrowing world of technology and nature. Variational autoencoders, local learning rules, layered learning, architectural choices, optimisers, loss functions, are a few of the many components surrounding this subject of machine learning that a profound understanding of was gained in the time spent working on solutions to the task at hand. The ability to comprehend, implement and even create at the level of programming would have seemed to be far too daunting just a few months before diving into this subject and its related topics.

Working with certain softwares, languages and programming libraries of which a small amount of prior experience acted as the sole preparation for was a difficult but most definitely rewarding task especially in a subject area with such deceptive nuance and depth. The skills gained from the pursuit of achieving the goals outlined in this report have made way for the capability to assist in the research of not just the topic at hand but have opened the door for many possibilities in similar realms of expertise.

In the search for a biologically feasible alternative to the standard backpropagation method present in almost all deep learning networks in use today, multiple different types of models, structures, and architectural choices have been explored and experimented with in order to gain the best possible understanding of the way in which the modification or creation of such models can allow for a realistic solution, or to even simply make progress towards the exposure of one.

Throughout this project various models and networks were both created and evaluated across multiple datasets of different sizes. Testing occurred on all kinds of aspects from tuning parameter values to assessing their ability from slight differences made while constructing from the ground up.

Results from this project show that a layered or staged training approach along with local learning rules in between layers and stages can produce a stable and structured latent representation, with the addition of a fairly accurate class-level organisation being created in an entirely unsupervised manner. The project discusses the extent to which this architecture achieves biological plausibility, acknowledges the differences from truly local learning, and highlights possibilities for future research.

Contents

1.	Background and Objectives	1
1.1.	Background	1
1.2.	Analysis	1
1.3.	Process	1
1.4.	Status of Work	1
2.	Design	2
3.	Implementation	3
4.	Testing	4
5.	Evaluation	5
	References	6
	Appendices	7
	A - Use of third-party code and libraries	7

List of Figures

...

1. Background and Objectives

1.1. Background

Deep learning has become the standard approach for computer vision tasks and almost all of it relies on backpropagation to train networks. Backpropagation is an algorithm that trains a neural network by computing the gradient of a global loss function with respect to every weight in the network, and using those gradients to iteratively update all layers simultaneously [1]. It works well in practice, but it has a significant problem in that it is not how a brain functions and therefore it has long been criticised regarding its lack of biological plausibility. There is a rising interest in discovering or understanding whether more biologically plausible alternatives could work just as well if not even better. However research into this area is still relatively growing and not many significant breakthroughs have been made as of late.

The core issue with backpropagation from a biological standpoint is that it requires neurons to receive very precise error signals that have been calculated using the exact same weights used in the forward pass, and propagated all the way from the output back to the input [2]. The brain has no known mechanism for doing this. Neurons send signals forward through synapses, but there is no equivalent backwards pathway carrying gradient information. There is in fact a name for this circumstance and it is referred to as the weight transport problem [2]. On top of this, backpropagation requires the whole network to stop and wait while the global error is computed before any weights can be updated. This, in contrast, does not mirror a brain's ability to learn continuously in real time.

The question therefore arises of whether artificial networks have the capability of using local learning rules (where each part of the network updates itself using only information that is directly available to it, without needing a global error signal). The oldest and most widely known example of this is Hebbian learning (a theory first proposed by Donald Hebb in 1949), which can be summarised by a term coined by Carla Shatz as "neurons that fire together, wire together" [3]. A connection between two neurons gets stronger when both are active at the same time, with no external supervision or error signal required. In practice, simple Hebbian rules on their own have been found to be nowhere near capable enough to train deep networks for complex tasks, but they do illustrate the fact that meaningful learning does not necessarily require a top-down error signal and can form from purely local interactions.

A more suitable approach to unsupervised or self-supervised learning is the Variational Autoencoder (VAE), first introduced by Kingma and Welling [4]. A VAE is a generative model that learns to compress input data (for example an image) into a compact latent representation and then decode and reconstruct the original input from that representation. The important part is that it does this without external labels and therefore is a form of self-supervised learning. The encoding is probabilistic, meaning that instead of mapping an input to a single point in latent space, the encoder outputs a mean and variance that define a distribution, and a sample is drawn from that distribution and passed to the decoder. The training process of this model aims to optimise two things at once: a reconstruction loss, which pushes the output to look like the input, and a KL divergence term, which keeps the latent space close to a standard normal distribution and well structured [4]. The result to aim for is a smooth latent space that the model can sample from to generate new data.

The brain processes visual information in a hierarchical and spatially organised manner. In the mammalian visual cortex, neurons in earlier areas like V1 (the primary visual cortex located in the occipital lobe) will respond to simple local features such as edges and orientations, while higher areas respond to more complex and abstract patterns [5]. Each layer builds upon the one below it and so higher level representations are built from lower level ones. Most importantly, each layer only directly processes input from a local region of the previous layer. Convolutional Neural Networks (CNNs) were in part inspired by this structure. They consist of local convolutional filters applied across an image to extract features at multiple different scales [6]. However, CNNs are still trained with global backpropagation from end to end, so the local structure of the computation does not reflect how the network actually learns.

A neuroscientific theory which corresponds to this is dubbed predictive coding, proposed by Rao and Ballard [7]. The main premise of this theory is that the brain is constantly making predictions about what it expects to see, and what actually gets passed between layers is the prediction error (the difference between what it expects and what it actually sees) rather than the raw input. This idea provides a biologically plausible explanation of how learning signals could be generated and used locally to update representations without the use of global backpropagation.

In machine learning, greedy layer-wise training [8] used in some Neural Networks takes a similar approach. In a deep network, one layer is trained at a time, and each layer learns to represent the output of the previous one before being frozen to allow the next layer to do the same. This avoids end to end backpropagation while still being able to produce deep representations in a hierarchical and local manner (just like biological learning).

This project brings all of these ideas together. The objective is to look for possibilities for more biologically equivalent methods of learning within artificial networks as well as conducting research into the options that already exist. Then the creation of a convolutional network that is trained progressively in a layer-wise manner, with each stage learning to encode and reconstruct a visual input while only using a locally self-supervised objective (such as a layer specific reconstruction or local loss) will be the end product. Most importantly it should be a system that reflects some of the biological principles described above while still being practical enough to produce high quality outputs and evaluations on standard vision benchmarks.

1.2. Analysis

The main challenge of this project is designing a system that can learn useful visual representations without relying on global backpropagation or external labels. It was glaringly paramount that the first step was to determine what the system actually needed to do in terms of processes and which existing approaches were the most relevant or useful to look into before commencing the construction of one.

The core requirements were as follows:

The system must be able to learn from raw image data without supervision.

It must be able to encode visual information into a compact, structured representation.

Learning should be guided by a local objective where possible (each part of the network should train on information available to it rather than depending on a global error signal).

Finally, the system should be able to be evaluated adequately, through producing outputs such as reconstructions, generated samples, and a structured latent space that can be inspected and compared.

The starting point was the creation and subsequent comprehension of a standard convolutional autoencoder. Autoencoders learn to compress and reconstruct their input, which satisfies the requirement of the system being unsupervised. Their convolutional structure also reflects the local, spatial nature of the data of images. A simple CNN with an autoencoder was the first build, as getting a firm grasp of the key encode to reconstruct pipeline was critical before delving into more complex variations. This CNN would also go on to provide a useful point of comparison for later models.

The main limitation of a plain autoencoder is that it produces a deterministic and often poorly structured latent space. Since the latent space is not guaranteed to be smooth or continuous, it's difficult to use it for generation or to recognise and draw conclusions about what the model has actually learned. A Variational Autoencoder on the other hand does not have these problems. By mapping inputs to standard distributions rather than fixed points and regularising the latent space with a KL divergence term, VAEs have the ability to produce a much more structured and interpretable latent space [4].

Once the VAE was established as the topic of my subsequent study into this topic, a significant amount of experimentation was then carried out to understand the effect of various design choices within such VAEs. All aspects of the model that were able to be tuned to produce a different output were thoroughly investigated and the effects of them were documented for later reference as the information gained while doing so would surely be crucial for problem solving and debugging with more complex networks. These aspects included optimisers (Adam, AdamW with weight decay, ... etc.), loss formulations (binary crossentropy, mse, mae, ...), activation functions (ReLU, sigmoid, tanh, ...), hyperparameters, regularisation, normalisation and augmentation methods etc.

The different weightings of the KL loss term relative to reconstruction loss are of very high importance when using a network which is affected by the KL divergence, since it has a direct effect on how structured the latent space becomes in comparison to how accurate the reconstructions are. It can be summed up by a simple formula that forms the basis to a Beta Variational Autoencoder's (β -VAE) framework [9]: $\text{Loss} = \text{Reconstructed_loss} + \beta * \text{KL_loss}$, where β at a value

of 1 will prioritise reconstructions and latent spaces equally, while higher values of β prioritise latent spaces and lower values prioritise reconstruction quality.

Training components such as customised callbacks which could be built through Keras and Tensorflow to serve a specific purpose were also meticulously explored and then introduced progressively into networks throughout the course of the project. Some of these included sigmoid KL annealing to prevent posterior collapse, learning rate scheduling with a warmup and cosine decay, and adaptive early stopping to prevent overfitting and help with resources and efficiency. Training was carried out across several datasets of increasing complexity, from randomly generated noise and simple grayscale databases like MNIST to more complex databases such as CIFAR-10 and CelebA in order to understand how different models and architectures behaved under different conditions.

The most significant design decision was deciding the form in which to introduce local, layered learning rules. After a sufficient comprehension of greedy layer-wise training [8] was obtained, it was clear that it had all the makings for a solid foundation upon which to develop a network which was consistent with the goals of the project. Instead of training all layers simultaneously, each layer or group of layers is trained independently before the network expands onto the next (groups of) layers. This approach could be adapted to fit the implementation of a VAE in conjunction, with the result being a progressive training scheme where the network grows one convolutional stage at a time with each stage only having the possibility to take in information collected during the previous stage. Each new stage is initialised by transferring weights from the one prior and then freezing the layers that have already been trained so that earlier representations cannot be affected by the layers training after as the network continues to grow. This is similar in concept to progressive training strategies used in generative modelling [10], and it aligns with the biological idea that lower level features in the visual hierarchy are learned and stabilised before higher level abstractions are then built on top of them.

An additional direction explored in this project was the incorporation of Hebbian learning, although not to a very large extent. Unlike the VAE's gradient-based updates, Hebbian rules update weights based purely on local pre and post synaptic activity, which represents a closer approximation to how synaptic plasticity operates in the brain [3]. Investigating whether a Hebbian component could be integrated into the architecture and what effect it would have on learned representations aligned perfectly with the project's biological motivation, although the absence of effectiveness that it has with no external involvement is a massive downside that can't be overlooked.

A collection of all of the elements related to the composition of artificial networks discussed in these last few paragraphs such as local, unsupervised learning rules, structural differences, architectural decisions and difference in standpoints or procedures will all be prominent factors addressed in the rest of this report.

1.3. Process

The project was developed in an iterative manner (with each iteration being each week) building directly upon the results of the previous one and leaving with set goals in mind for the next iteration. Rather than rashly diving head first into the objectives that were set and trying to produce a full system at once, the approach taken was to start with the simpler networks and get familiarised and comfortable working with them. After feeling that sufficient knowledge in one level of networks was gathered only then it would be time to look into more complex networks and topics. This approach albeit a little slow due allowed for an easier time solving and understanding problems which arose throughout the creation and modification of these networks, most importantly when faced with first time issues as a lot of the knowledge that would be lost if some tasks were to be rushed were able to be translated between aspects all over the subject at hand.

The bulk of development was done using Visual Studio Code in Python with the help of the TensorFlow and Keras libraries for the majority of machine learning tasks along with Matplotlib and NumPy for data visualisation. Visual Studio Code had to be run inside of a WSL2 environment on a Windows 10 machine with a dedicated (medium to high end) GPU with 8GB of VRAM. The use of a Windows Subsystem for Linux (WSL2) was only necessary due to the fact that Tensorflow with GPU compatibility's support is quite poor for native Windows users and the only versions available are outdated and error prone. Resorting to the usage of the CPU only version would not have given me enough computing power to complete many of the tasks that were essential. The same can be said about using a Virtual Machine to go around this limitation. Up to date versions of the CudNN and CUDA libraries are also a prerequisite for Tensorflow with GPU support on all operating systems. Regarding the programming software, Visual Studio Code along with Jupyter Notebook and some quality of life extensions alleviated a lot of annoyances throughout the coding process. Jupyter suited the experimental nature of the work very well since individual cells could be run and modified independently without having to re-run the entire pipeline each time. Singling out lines or chunks of faulty code and debugging in general is a much easier task when code is split up in this manner as well.

The general process of the project was based on meeting weekly objectives and iterations as stated before. Reading up on Neural Networks and the creation of a simple Convolutional Neural Network with an autoencoder was the first of the tasks that I carried out while looking to gain experience and further insight into the subjects surrounding the project at hand. I then extended that network with an implementation of a simple VAE. The VAE was refined over a number of iterations with all possible elements being looked at, experimenting with different training configurations, loss formulations, and callbacks to find the combinations of methods or values for hyperparameters which gave the best outputs.

The models were then tested on progressively more complex datasets, starting from random pixelated images of pure noise from NumPy and small scale databases like MNIST before moving on to more complex datasets like CIFAR-10 and CelebA. This progression was deliberate, as simpler datasets made it easier to spot problems early without the added difficulty of more complex image distributions, not to mention the drastic increase in computation time making it a much longer process too.

Once the ceiling of quality of outputs in comparison to the capacity of the VAE model had been reached, the focus then of course had to shift to structural and architectural changes, as the only

way to improve was to expand the framework and size of the model. One such way of doing this was adjusting the network by using the progressive layer wise training scheme that was also a step further towards biological learning methods. That is after all essentially the core premise of the project, the shift away from backpropagation and towards local learning rules. Hebbian learning components were also looked into and played around with as a possible step toward biological plausibility as well, although not to much success. The addition and usage of a local loss function was also a massive step forward not only just in terms of a biological solution but with output quality too.

Throughout the project, results were evaluated visually using reconstruction plots to assess reconstruction quality, generated samples for generations, and latent space PCA for class level organisation, as well as quantitatively through training and validation loss curves and values, most of which the usage of Matplotlib was employed. Version control and documentation was maintained in a GitHub repository.

1.4. Status of Work

The core objectives and proposed tasks of the project were successfully completed. Research into self-supervised and unsupervised learning in Deep Neural Networks (DNNs), VAEs and other models, biological and local learning methods and alternatives to backpropagation were completed to a high standard.

A successful implementation of an algorithm with a hierarchical and convolutional composition, along with local training objectives including the use of local loss weighting was achieved in the form of a layered, easily adjustable and transparent autoencoder network. The network created outputs a spatially well structured latent space, with a balance of good reconstructions and generations once the right balance of parameters are found. Said parameters were also made easily adjustable right at the top of the code assigned to constants.

There was sufficient development and thorough evaluation conducted and all models were tested across multiple datasets. Knowledge on different network configurations, architecture, the effect of parameters, activation functions, normalisation and regularisation techniques etc. were greatly broadened throughout the duration of the project. The codebase is functional, documented, easily available and reproducible. Ultimately, the project deliverables defined at the beginning have been achieved.

The Hebbian learning component was not implemented and integrated into the architecture, although it was tested briefly to the extent of which will be discussed in the Evaluation section. Given the time constraints of the project, that part of the work should be treated more like an exploratory extension than a fully developed contribution.

Some areas did not go as far as originally planned. A more comprehensive evaluation on Hebbian learning and a complete implementation and exploration of such rules would have been ideal, but unfortunately there was not enough time to do so fully and with the little time that I did have to play around with Hebbian learning rules they were more than disappointing in terms of results.

Likewise, experimenting and further testing on higher resolution datasets beyond CelebA was also considered but ultimately out of scope given the available time and resources available. With a single decent run through CelebA with just a VAE already taking almost a quarter of an hour (not to mention the time taken to download the database itself) it would have been far from wise to venture into testing on larger, more complex datasets as it would have been a massive time sink if I could even manage to train on it with seriously minimal returns regardless.

Overall I do believe the project is in a complete and satisfactory, submittable state. The implementation meets the requirements set out at the start of the project, and there is enough experimental data to support a meaningful evaluation and discussion as well as a solid base for further research to be made with.

Objective	Complete	Notes
Fully functional implementation of a computer vision algorithm (e.g. autoencoder)	Yes	
Implementation of algorithm includes the use of local training objectives	Yes	
Implementation of algorithm learns in an unsupervised / self-supervised manner	Yes	
Implementation of algorithm includes a convolutional or hierarchical composition	Yes	
Implementation of algorithm avoids backpropagation to a suitable extent	Yes	Avoiding backpropagation entirely will always result in a massive drop in efficiency or quality
Implementation of algorithm makes use of a Variational Autoencoder	Yes	
Implementation of algorithm learns with Hebbian learning rules	Partial	I've made CNNs which learn with Hebbian learning but are also extremely simple with dissatisfactory and low quality outputs
Implementation of algorithm is easily customisable for desired outputs	Yes	
Implementation of algorithm produces high quality outputs	Yes	
Thorough research conducted into Convolutional Neural Networks	Yes	
Thorough research conducted into Variational Autoencoders	Yes	
Thorough research conducted into existing alternatives to backpropagation	Yes	
Thorough evaluation done on efficiency and output of algorithm	Yes	
Kept GitHub repository up to date throughout the project's duration	No	
Created iterative planning / roadmap on GitHub	Yes	
Source code and documentation providing instructions on reproducing results hosted in a repository such as GitHub	Yes	

2. Design

To preface this section, I will be describing the network that was the last updated and what I believe to be the one with the most relevance to the expected end product of the project, as this specific network features a greedy layer-wide, hierarchical and convolutional structure, along with a local loss function for all encoder blocks (with only the decode and bottleneck being affected by the global loss function) and naturally it learns self-supervised with a VAE implementation. As it checks a lot of the boxes that most would consider deeming a network to be more biologically plausible, I've decided to pick this one to describe over the others. This one checks the most boxes out of the ones I've created and fits the project outline the best, as most of my others don't have a working loss function yet. However I have achieved better reconstructions and generations with a lot of my other networks, as well as used them on more complex datasets with better results, although this one still also does produce the most structured latent space out of all of them.

2.1 Overall Architecture

The system is built around a convolutional Variational Autoencoder. The fundamental design of all Variational Autoencoders consists of an encoder, which compresses an input image into a latent representation, and a decoder, which reconstructs the image from that representation. Both are made up of convolutional blocks, which are perfect for image data since convolutions process local spatial regions rather than the entire input at once.

The encoder however is implemented through a Class called LocalEncoder(keras.Model) instead of a usual build_encoder function. This allows the system to operate using local learning rules due to the fact that you can fit `tf.stop_gradient()` in a subclassed model between blocks during the forward pass, which is not directly possible otherwise in a regular model. `tf.stop_gradient()` is a tensorflow operation which stops the flow of gradients, and when placed between blocks like so it prevents the gradient of a higher layer from affecting any layer below it, therefore enforcing local learning at least for all the encoder blocks. There will be more elaboration under section 2.3.

Keeping the latent representation spatial rather than flattening it into a vector is a huge design change as in a typical VAE the encoder flattens everything down to a one dimensional vector, losing all positional information in the process. In this case it's just a 4x4 grid of latent vectors instead, where each position in the grid corresponds roughly to a region of the original image. This keeps the latent space better organised and is more consistent with the idea of local representation, since different parts of the latent space are responsible for different parts of the image rather than everything being mixed together in a single flat vector.

2.2 Encoder and Decoder Blocks

Each convolutional block in the encoder consists of two convolutions which happen one after the other, the first being a 3×3 convolution that extracts local spatial features from the input. Its stride length of 2 halves the spatial dimensions where downsampling is required. The second is a 1×1 convolution that performs channel mixing, recombining the resulting feature channels without affecting spatial dimensions. The 1×1 convolution allows the network to learn a little more for a low computational cost. ReLU activations follow each convolution. The number of filters doubles with each successive block so blocks deeper in the network have better odds of capturing more complex and abstract features.

The decoder mirrors the structure but in reverse. Each decoder block begins with a 1×1 convolution for channel projection, followed by a transpose convolution that upsamples the spatial dimensions back toward the original resolution. The filter counts decrease symmetrically with depth, ending with a final 3×3 convolutional layer that maps back to the number of image channels. A tanh activation is used at the output, which restricts pixel values to the range $[-1, 1]$, matching the normalisation applied to the input data.

The final encoder layer outputs two spatial maps, one for the mean (z_mean) and one for the log variance (z_log_var) of the latent distribution. These are kept in full float32 precision even when mixed precision training is used elsewhere, since numerical stability in the latent space is important for the KL loss computation.

2.3 Local Self-Supervised Objective

The most crucial part of this architecture is that the encoder does not wait on or use a global error signal to train its lower layers. Each convolutional block is equipped with a small local prediction head which is a small convolutional network with two layers that takes the current block's output and tries to reconstruct the output of the block below it. The loss from this local head is computed and applied to every block, but in this case since a stop-gradient operation is inserted between every encoder block this means each block only receives training signal from its own local objective, not from the VAE loss further up the network. The VAE loss at the bottleneck trains only the innermost block and the latent projection layers. This exactly mirrors the predictive coding framework proposed by Rao and Ballard[7], where higher and further back areas of the brain predict lower area (V1, occipital lobe) activity and as a consequence only the prediction error is passed between them instead of the raw signal.

The local heads are designed to account for spatial dimensions between adjacent blocks. Where the encoder downsampled with a stride of 2, the head uses a transpose convolution to upsample before computing the error. Where adjacent blocks share the same spatial size, a standard convolution is used instead.

The local loss is weighted by a scalar hyperparameter `LOCAL_WEIGHT` (assigned to a constant at in the first cell) and summed across all active blocks before being added to the total loss for that training step. This makes it straightforward to control the contribution of the local objective relatively to global VAE terms.

2.4 Progressive Layer-Wise Training

The network is built in increments across for example 4 stages. Stage one trains a single encoder block and its relative decoder. Each subsequent stage adds one more block on top, so by stage four the full depth of the network is reached. This greedy layer-wise approach means each new stage only needs to learn the additional complexity it introduces, instead of picking up everything from scratch at once.

At the start of each stage, weights from the previous model are transferred into the new larger model by matching layer names through a function called `transfer_weights()`. The already-trained layers are then frozen by a function called `freeze_outer()` so that adding a new stage cannot overwrite what was previously learned. In the encoder, the blocks closest to the input are frozen since they represent the most stable low-level features. In the decoder, the blocks closest to the output are frozen for the same reason. The new trainable region at each stage is always the innermost part of the network, usually the bottleneck along with a newly added block.

This has a direct parallel to the visual cortex, where low-level features like edges are established early and remain stable while higher-order representations continue to develop on top of them, once again what Rao and Ballard spoke of, predictive coding [7]. It also has a practical benefit in that only a small portion of the model is trainable at any one stage, which reduces memory usage and speeds up convergence.

2.5 Loss Function Design

The total training objective combines three terms: reconstruction loss, KL divergence, and the sum of local prediction losses from section 2.3.

Reconstruction loss is mean squared error (MSE) computed over the full image, summed across pixels per image then averaged across the batch. MSE was chosen over binary cross-entropy because CIFAR-10 images are natural photographs with continuous pixel values, and MSE is better suited to this than a loss function designed for binary outputs in the range of $[0, 1]$.

The KL divergence term regularises the latent space by penalising the encoder for producing distributions that deviate from a standard normal prior. This keeps the latent space smooth and structured, which benefits the quality of generations greatly. Without it, the encoder tends to produce an irregular or collapsed latent space. The relative weighting follows the β -VAE formulation[9], where total loss = reconstruction loss + β * KL loss, with β set to 1.0.

To prevent posterior collapse (when the encoder ignores the KL term due to its value either being extremely low or high, resulting in the latent space becoming informatively useless) KL annealing is applied (discussed further in section 3.6). The KL weight starts near zero at the beginning of each stage and rises smoothly to its target following a sigmoid curve. This gives the reconstruction objective time to establish a useful signal before KL pressure is introduced, and tends to produce more stable results than either a fixed weight or a linear schedule.

2.6 Training Configuration

The optimiser used is AdamW, which adds decoupled weight decay to Adam as a mild regularisation (making L2 regularisation obsolete as it already comes with it). Gradient clipping (an adjustable parameter for AdamW) is also applied by default to prevent instability in early epochs when loss values are large.

The learning rate follows a cosine decay schedule with a short linear warmup at the start of each stage. A fresh schedule is created by recompiling the optimiser function at the beginning of every stage, resulting in starting off each stage with the same peak learning rate regardless of how the previous stage ended. This is very important as the newly added block needs a high learning rate to learn from scratch, and if it were to continue with the previous stage's decayed schedule it would leave it under-trained.

Early stopping uses a custom adaptive callback (more on this in section 3.6). Unlike standard early stopping with a fixed patience that has to be changed whenever the losses change in scale significantly, this customised version reduces its patience dynamically whenever the learning rate drops significantly, since a sharp drop usually signals the model is close to convergence already. Early stopping is also delayed for the first half of each stage to avoid terminating too soon before the cosine schedule has had time to take effect.

2.7 Data Pipeline

The dataset used here is the CIFAR-10 dataset, consisting of 60,000 32×32 colour images across 10 classes. Images are normalised to the range $[-1, 1]$ to match the tanh output activation of the decoder (used in the output layers of this network).

Augmentation (a function which comes with TensorFlow) is applied during training using random horizontal flips, small brightness jitter, and slight contrast variation. This is done on the fly within the data pipeline rather than pre-processing in advance, so each epoch sees slightly varied versions of each image. This in turn reduces the risk of overfitting. The validation set uses no augmentation so that validation metrics are consistent and comparable across all stages.

3. Implementation

3.1 Sampling Layer and the Reparameterisation Trick

The VAE requires sampling from the latent distribution during the forward pass, but a random sampling operation has no gradient and cannot be trained through directly. The reparameterisation trick resolves this by rewriting the sample as a deterministic function of the distribution parameters plus a separate noise term. Rather than sampling z directly from the distribution defined by z_mean and z_log_var , a random noise vector ϵ is drawn from a standard normal distribution and the sample is computed as $z = z_mean + \exp(0.5 * z_log_var) * \epsilon$. The randomness is now in ϵ , which is not a learned parameter. This then allows gradients to flow through z_mean and z_log_var as normal during backpropagation.

This is implemented as a standalone Keras layer or Class called Sampling.

3.2 Encoder

The encoder (in this network) is implemented as a subclassed Keras model called LocalEncoder rather than a functional model. This was required because the local learning scheme requires a stop-gradient operation to be inserted between blocks during the forward pass, which is usually impossible in a functional model.

Each block is constructed as a small sequential model containing a 3×3 convolution followed by a 1×1 convolution, both with ReLU activations. The blocks are stored in a list and iterated over in the call method. After each block's output is computed, `tf.stop_gradient` is applied before passing it to the next block. This is the line that stops the backwards pass and makes the encoder truly locally self-supervised. Without it, the VAE loss at the bottleneck would backpropagate through every block regardless of the local heads.

The bottleneck sits after the final block and consists of two separate 3×3 convolutions producing `z_mean` and `z_log_var` as spatial maps, followed by the Sampling layer. Both bottleneck convolutions are cast to float32 explicitly, even when the rest of the network runs in float16 under mixed precision (for better GPU performance, more on this at section 3.7), since the KL loss involves exponentials and squared terms that are numerically unstable at lower precision.

The call method returns `z_mean`, `z_log_var`, and `z` as the first three outputs, followed by the activation of every block in order. These activations are used by the local heads in the VAE's training step. Because LocalEncoder is a subclassed model, it does not have an `outputs` attribute like a functional model would.

3.3 Decoder

The decoder is built as standard, through a functional Keras model. It takes the spatial latent sample z as input and progressively upsamples it back to the original image resolution. Each block mirrors the encoder in reverse, starting with a 1×1 convolution for channel projection followed by a transpose convolution for upsampling. Filter counts decrease symmetrically with depth. A final 3×3 convolution maps back to the number of image channels with a tanh activation.

3.4 VAE and Training Step

The VAE is a subclassed Keras model that holds the encoder, decoder, and local prediction heads as attributes. The local heads are built inside `__init__` by iterating over adjacent block pairs and calling `build_local_head`, which constructs a small two-layer convolutional network. Whether the head uses a transpose convolution or a regular convolution depends on whether the corresponding encoder block downsamples.

The training step uses a custom `train_step` method through `tf.GradientTape`. Inside the tape, the encoder is called to get `z_mean`, `z_log_var`, `z`, and all block activations. The decoder is then called on `z` to produce the reconstruction. The VAE loss is computed from the reconstruction and KL terms. The local loss is computed separately by iterating over adjacent activation pairs, passing each through its corresponding local head and computing the MSE (mean squared error) against the stop-gradient target. The two losses are combined and gradients are computed over all trainable weights in a single call.

All intermediate tensors involved in loss computation are explicitly cast to `float32`. This is necessary because mixed precision (which when enabled boosts performance, more about this at section 3.7) training causes activations to be stored in `float16`, and combining `float16` and `float32` tensors without explicit casting produces silent type errors that can cause incorrect gradient computation.

Four metrics are tracked throughout training: total loss, reconstruction loss, KL loss, and local loss. The local loss is tracked in both train and validation steps so that it appears properly in the training history and can be plotted at each stage.

3.5 Weight Transfer and Layer Freezing

When a new stage begins, weights from the previous stage's encoder and decoder are transferred into the newly built models using a self created function called `transfer_weights`. This function keeps track of the old model's layers names then iterates over the new model's layers, looking each one up by name and copying its weights if a match is found. Layers are skipped if they are not present in the old model, if the weight shapes do not match, or if they are put in a skip list (such as `z_mean` and `z_log_var`, since the bottleneck is intentionally re-initialised at each stage it would be pointless to iterate over them).

Name-based matching was chosen over index-based matching because the two models have different total layer counts at each stage, so the same logical layer will appear at a different index in the new model. Matching by name works out to be much more reliable in my experience and makes it easy to see which layers exactly were transferred and which were skipped from the debug output.

Layer freezing is handled by `freeze_outer`, another self created function, which sets `trainable = False` on the appropriate encoder and decoder layers by storing prefixes of layers it needs to freeze and iterating through a set variable to freeze the layers that need freezing.. In the encoder, the blocks closest to the input are frozen. In the decoder, the blocks closest to the output are frozen. At stage `N`, the `N-1` outermost blocks on each side are frozen, leaving only the innermost block and the bottleneck as trainable.

3.6 Callbacks

KL Annealing was created as there wasn't a callback in neither Keras nor Tensorflow despite its popularity in usage, therefore I had to create my own.

The other two are modified versions of existing callbacks but improved to save compute power, time, increase overall efficiency as well as output quality.

KL Annealing:

The KLANneal callback updates the KL weight variable at the start of each epoch following a sigmoid schedule. A sigmoid curve was chosen over a linear ramp because it rises slowly at first, accelerates through the middle, and then flattens out again near the target value. This behaviour is more stable than a linear schedule since it avoids an abrupt jump in KL pressure that can destabilise training if the reconstruction loss has not yet converged. The callback tracks a global epoch offset to ensure the annealing curve continues correctly across stages rather than resetting to the beginning each time.

AdaptiveEarlyStopping was created as an alternative to the EarlyStopping callback as the parameters would have to be adjusted every time the losses drastically rose or sank. One of its features includes reducing its patience dynamically whenever the learning rate drops by more than a set threshold RELATIVE to the previous epoch instead of a set number. A significant LR reduction usually means the cosine schedule has entered its decay phase and the model is approaching convergence, so waiting the full original patience afterwards is unnecessary. The callback also has a configurable start epoch so that early stopping is suppressed for the first portion of each stage, preventing premature termination when the losses are all over the place at the beginning of each stage and before the cosine scheduler has had time to take effect.

EfficientModelCheckpoint when it was first created started saving checkpoints a little bit after the set number of epochs I'd arrange for AdaptiveEarlyStopping to start after. However, since then I realised that because AdaptiveEarlyStopping saves the best weights anyway, I can allow this callback to do nothing at all until the end of training sends a trigger for it to just take the best weights AdaptiveEarlyStopping has saved and then save it to a file. This also avoids the extra compute power of repeated file writes during training while still ensuring the best weights are preserved regardless of whether early stopping was triggered. The constant messages cluttering up the cell output won't be missed either.

3.7 Mixed Precision

Mixed precision training is enabled globally using Keras's `mixed_float16` policy. To enable it you simply put this line at the start of your code:

```
keras.mixed_precision.set_global_policy('mixed_float16')
```

This causes most layer computations to run in float16, which roughly halves memory usage and increases throughput on supported GPUs, while keeping a float32 copy of the weights for the optimiser update. The practical consequence for this architecture is that all encoder and decoder activations, and the local head outputs, are in float16 by default.

The exception is the latent bottleneck. The `z_mean` and `z_log_var` layers are declared with `dtype='float32'` explicitly, which forces their outputs to remain in full precision. This matters because the KL loss computes $\exp(z_log_var)$ and z_mean^2 , both of which can overflow or underflow at float16 range for moderately large values. All loss computations in `_compute_losses` and the training step also cast their inputs to float32 explicitly before any arithmetic is performed.

Often I'll find that so many values have to stay in float32 otherwise the program will refuse to run, and you'll have to manually cast it back to float32 through `dtype` or `tf.cast()`, which will eventually end up getting on your nerves.

4. Testing

4.1 Component Verification

The first level of testing was verifying that each component of the architecture behaved correctly before running a full training stage. At the start of each stage, the encoder and decoder are built and a dummy forward pass is performed on a zero-filled batch of the correct input shape. This forces both models to build and allows their output shapes to be inspected before any training begins. Any shape mismatch between the encoder output and the decoder input would raise an error at this point rather than partway through training, making it easier to catch architectural mistakes early and saving a lot of time and frustration.

The trainable parameter count is also printed before each stage begins, along with the names of all frozen and trainable layers in both the encoder and decoder. This makes it straightforward to verify that the freezing logic is working as intended, since the list of trainable layers should shrink at each stage as more blocks are frozen, and any unexpected entry in that list would indicate a bug in `freeze_outer`.

Similarly, the weight transfer function prints the number of successfully transferred layers and any layers that were skipped along with the reason, either a shape mismatch or an explicit skip. This output was checked at each stage to confirm that previously trained weights were being carried over correctly into the new model.

If there was a problematic variable that sporadically would raise errors, printing it out in the cell output constantly was also always a surefire way to get closer to solving the underlying issue.

4.2 Training Behaviour

Once a stage was running, the primary indicator that training was proceeding correctly was the behaviour of the loss curves. For each stage, total loss, reconstruction loss, KL loss, and local loss were tracked separately across every epoch and plotted at the end using Matplotlib. A correctly behaving run shows a sharp drop in total and reconstruction loss in the first few epochs, followed by a smooth convergence.

Metrics detailing every single loss value at every epoch of every stage in its own separate cell at the end was also quite helpful with debugging.

The KL loss should rise gradually from near zero during the annealing warmup and then stabilise, rather than spiking or collapsing to zero, either of which would indicate a problem with the annealing schedule or a posterior collapse.

Both the learning rate and KL_weight were also printed at the start of each epoch, the learning rate via a self created Lambda callback, which made it easy to verify that the cosine decay schedule was progressing as expected and that the warmup was functioning correctly at the beginning of each stage. The same for KL_weight and making sure its sigmoid annealing was working properly.

The AdaptiveEarlyStopping callback also prints a message each time it reduces its patience or decides to stop, providing a readable trace of why training ended when it did.

4.3 Reconstruction Quality

After each full training run, reconstruction quality was assessed visually by passing a fixed set of ten test images through the encoder and decoder and plotting the originals alongside the reconstructions. A well-trained model should produce outputs that closely match the input in terms of colour, shape, and overall composition, even if fine detail is lost due to the compression. Blurry or washed out reconstructions indicate that the reconstruction loss is too weak relative to the KL term, while reconstructions that are sharp but inconsistent with the input suggest the latent space is under-regularised.

This plot was used throughout development to catch regressions and to assess whether changes to the architecture or hyperparameters had a positive or negative effect on output quality.

4.4 Generative Quality

Generation quality was assessed by sampling random vectors from a standard normal distribution with the same shape as the latent space and passing them directly through the decoder. The resulting images were plotted in the same way as the reconstructions. Unlike reconstructions, which are anchored to a real input, generations depend entirely on the structure of the learned latent space. A well-regularised VAE should produce outputs that are visually coherent even from a random sample, while a poorly structured latent space will produce noisy or incoherent outputs regardless of decoder quality.

This distinction between reconstruction and generation quality is an important one. A model can achieve excellent reconstruction loss while still producing poor generations if the latent space is not sufficiently smooth and regular. Monitoring both outputs separately throughout training made this easier to diagnose.

4.5 Latent Space Structure

The structure of the learned latent space was assessed using Principal Component Analysis (PCA). After training, the encoder was run on the full CIFAR-10 test set to produce a latent map for each image. These maps were flattened into vectors and projected down to two dimensions using a manual PCA implementation in NumPy. The resulting 2D points were coloured by class label and plotted as a scatter plot.

A well-structured latent space should show some degree of class-level organisation, with images from the same class clustering together, even though the model was trained with no class labels at any point. The degree of separation between clusters gives an indication of how much semantically meaningful structure the model has captured in an entirely unsupervised manner. Overlapping or uniformly distributed points would suggest the latent space has not organised itself around meaningful features.

5. Evaluation

5.1 Objectives Met

The core objectives set out at the start of the project were successfully completed. A convolutional VAE with a spatially structured latent space was designed, implemented, and evaluated. The progressive layer-wise training scheme was fully realised, with weight transfer and layer freezing working correctly across all four stages. The local self-supervised objective was integrated into the encoder through per-block prediction heads and stop-gradient operations, meaning the encoder genuinely does not rely on a global error signal to train its lower layers.

All models were evaluated across multiple datasets of increasing complexity, from NumPy noise and MNIST through to CIFAR-10 and CelebA, producing reconstruction plots, generation samples, loss curves, and a latent space PCA at each stage of development. The codebase is functional, documented, and reproducible.

The Hebbian learning component was explored separately as an extension but was not integrated into the final model. Given the time constraints of the project this was a reasonable decision, and it is treated here as exploratory work rather than a failed objective, especially due to how poorly it lines up against others on its own. A deep dive into integrating it into a small part of a functional network would've been very interesting to look at though.

5.2 What Went Well

The progressive training scheme worked reliably throughout. Loss curves at every stage showed smooth and stable convergence, and the weight transfer and freezing mechanism successfully preserved earlier representations without degradation as new stages were added. The KL annealing schedule prevented posterior collapse in all runs, which was a significant practical success given that collapse is a common failure mode in VAE training and one that took several iterations of the annealing design to solve reliably.

Reconstruction quality was strong by the end of the project, with the final model producing visually faithful outputs across a variety of CIFAR-10 classes. The iterative development process, starting from a simple autoencoder and gradually adding complexity, proved to be an effective strategy. Problems were caught and understood early on simpler datasets where they were easier to diagnose, and the knowledge accumulated through that process made later stages of development significantly faster.

The decision to implement custom training components rather than relying entirely on built-in Keras utilities also paid off. The adaptive early stopping, sigmoid KL annealing, and efficient model checkpoint callbacks gave much finer control over training behaviour than standard callbacks would have allowed, and the transparency of their printed output made debugging considerably easier.

5.3 What Could Have Been Done Better

References

[1] Y. LeCun, "A theoretical framework for back-propagation," in Proc. Connectionist Models Summer School, D. Touretzky, G. Hinton, and T. Sejnowski, Eds., Pittsburgh, PA, USA: Morgan Kaufmann, 1988, pp. 21–28.

- An early theoretical treatment of backpropagation, cited to establish the foundational definition of how backpropagation works and what it requires before discussing its biological limitations.

[2] T. P. Lillicrap, A. Santoro, L. Marris, C. J. Akerman, and G. Hinton, 'Backpropagation and the brain', *Nat Rev Neurosci*, vol. 21, no. 6, pp. 335–346, Apr. 2020, doi: 10.1038/s41583-020-0277-3.

- A review paper covering the biological implausibility of backpropagation and the weight transport problem, which provided the core theoretical motivation for moving toward local learning rules.

[3] R. G. Morris, 'D.O. Hebb: The Organization of Behavior, Wiley: New York; 1949', *Brain Res Bull*, vol. 50, no. 5–6, p. 437, 1999, doi: 10.1016/s0361-9230(99)00182-3.

- A citation to Hebb's original 1949 work on synaptic plasticity, which grounded the project's discussion of Hebbian learning as a biologically plausible alternative to gradient-based updates.

[4] D. P. Kingma and M. Welling, 'Auto-Encoding Variational Bayes', Dec. 20, 2013, arXiv:arXiv.1312.6114. doi: 10.48550/arXiv.1312.6114.

- The original VAE paper, which introduced the reparameterisation trick and the combined reconstruction plus KL divergence objective that forms the foundation of this project's architecture.

[5] D. H. Hubel and T. N. Wiesel, 'Brain Mechanisms of Vision', *Sci Am*, vol. 241, no. 3, pp. 150–162, Sep. 1979, doi: 10.1038/scientificamerican0979-150.

- A landmark study on the hierarchical organisation of the mammalian visual cortex, used to support the biological motivation for building a hierarchical, locally operating convolutional architecture.

[6] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998, doi: 10.1109/5.726791.

- The foundational paper on convolutional neural networks, relevant as the basis for the convolutional structure used throughout the encoder and decoder.

[7] R. P. Rao and D. H. Ballard, 'Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects', *Nat Neurosci*, vol. 2, no. 1, pp. 79–87, Jan. 1999, doi: 10.1038/4580.

- The original predictive coding paper, which directly inspired the local prediction head design where each block learns to predict the representation of the block below it.

[8] Y. Bengio, P. Lamblin, D. Popovici, and H. Larochelle, 'Greedy Layer-Wise Training of Deep Networks', in *Advances in Neural Information Processing Systems*, MIT Press, 2006. Accessed: Apr. 21, 2026. [Online]. Available:

<https://proceedings.neurips.cc/paper/2006/hash/5da713a690c067105aeb2fae32403405-Abstract.html>

- Introduced greedy layer-wise training of deep networks, which is the direct precedent for the progressive staged training scheme implemented in this project.

[9] Higgins *et al.*, ‘beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework’, Feb. 2017. Accessed: Apr. 22, 2026. [Online]. Available:

<https://openreview.net/forum?id=Sy2fzU9gl>

- Introduced the β -VAE framework, which formalised the idea of weighting the KL term relative to reconstruction loss and informed the KL weight tuning carried out during development.

[10] T. Karras, T. Aila, S. Laine, and J. Lehtinen, ‘Progressive Growing of GANs for Improved Quality, Stability, and Variation’, Feb. 26, 2018, *arXiv*: arXiv:1710.10196. doi: 10.48550/arXiv.1710.10196.

- Introduced progressive growing of GANs, cited as a related example of progressive training strategies in generative modelling that influenced the staged approach taken here.

Appendix A - Use of Third-Party Code and Libraries

- Visual Studio Code
- Python
- Jupyter Notebook
- TensorFlow
- Keras
- Matplotlib
- NumPy
- CUDA
- cuDNN

Appendix B - Hyperparameter List

```
LATENT_DIM      = 128
KL_WEIGHT       = 1.0
KL_STEEPNESS    = 0.2
LOCAL_WEIGHT    = 0.1
BATCH_SIZE      = 512
BASE_FILTERS    = 16
LEARNING_RATE   = 0.001
LR_DECAY        = 0.75
WEIGHT_DECAY    = 0.0005
CLIPNORM        = 2.0
EPOCHS          = 250
BOTTLENECK_DIM  = 4
MAX_LAYERS      = 5
LOSS_FUNCTION   = str('mse')           # 'mse' or 'bce'
ACTIVATION_OUT  = str('tanh')          # 'tanh' or 'sigmoid'

IMAGE_SIZE_X    = 32
IMAGE_SIZE_Y    = 32
CHANNELS        = 3                    # 1 for grayscale, 3 for RGB
STRIDE          = 2

STAGE_EPOCHS    = int(EPOCHS // MAX_LAYERS)
WARMUP_EPOCHS   = int(STAGE_EPOCHS // 10)
AES_PATIENCE    = 10
AES_SENSITIVITY = 0.075                # Lower is more sensitive
AES_MIN_DELTA   = 0.01
AES_START_EPOCH = int(STAGE_EPOCHS * 0.5)
```
